

Martian Weather Trends Study

Vérin Clément
ISIMA, ZZ2 F1

Trundle Paul
Bradford University

May 8, 2025



Figure 1: Selfie of the *Curiosity* Rover (October 2021)

[Deposit Link](#)

Summary

1	Introduction	3
2	CSV File Description	3
3	First view on Statistics	3
3.1	Column Selection.....	3
3.2	Parameters Evolution According to Solar Longitude	4
3.3	Parameters Evolution Sol per Sol.....	6
4	Clustering	7
4.1	Cluster Training	7
5	Predicting the Weather ?	12
5.1	LSTM Adapted CSV Maker Script	12
5.2	<i>Weka</i> adapted LSTM strategy	13
5.3	Final Results and Testing Phase.....	15
6	Conclusion	17
7	References	17

1 Introduction

On August 2012 the 6th, the Rover *Curiosity* landed in *Gale's Crater* on *Mars*. Despite sandstorms, dust, radiations, brutal temperature drops and raises *Curiosity* is still working today. Thanks to *Curiosity* and the other Rovers on *Mars*, we have been able to daily sample and measure the weather on *Mars*.

The main goal here is to give an overview of the provided *mars-weather* CSV file, to deploy its content and to use it, not exactly in order to "*predict*" the weather on *Mars*, but rather in order to extract a general behaviour and cycle of the weather by looking at the given conditions. Since *Curiosity* has not moved very far from its starting point, we can consider its location as still.

2 CSV File Description

The *mars-weather* describes 10 columns which are:

id: Unique number representing the sample
terrestrial_date: Earthian date of sample collection.
sol: Number of Marsian days elapsed since the *Curiosity* Rover landed. One "day" on Mars last 24h39, wich is called a "sol".
ls: "Solar Longitude". Position of mars on its orbit around the sun. That's why it moves between 0 and 360 degrees.
month: Earthian month of sample collection.
min_temp: Minimal temperature noted on this sol.
max_temp: Maximal temperature noted on this sol.
pressure: Pressure noted on this sol.
wind_speed: Speed of the wind noted on this sol.
atmo_opacity: General weather of this sol.

3 First view on Statistics

3.1 Column Selection

First, let's ignore some informations in order to focus on the most useful ones.

The columns **id**, **terrestrial_date**, **month**, **wind_speed**, and **atmo_opacity** are not relevant for us because they are either not directly useful for statistics, either not returned, or etither unchanged through instances.

In this Coursework, I personally thinks it would be easier to count the time as we were on *Mars*, because one day duration on *Mars* is pretty close to *Earth*, and *Mars* is the actual studied planet here.

So one *day* becomes one *sol*, which is a very important information, allowing us to track the weather through sols.

Moreover, a year on Mars is approximatively 668 sols (687 days on *Earth*).

Talking about weather behaviour throughout a Martian year, the first obvious parameter we could think about is *Solar Longitude*: since *Mars* has a not

null rotation axis like *Earth*, *Mars* certainly has four seasons which impacts its weather.

By the way, this angle is 23.5 degrees on *Earth* and 25.19 degrees on *Mars*, which are not so different.

Of course, **min_temp**, **max_temp** and **pressure** are the statistics that reflect the current weather on *Mars*, so they are the main studied columns.

3.2 Parameters Evolution According to Solar Longitude

Let's plot the evolution of these 3 parameters by sol using Python libraries. You can find the source code in the *Python* repository as *Mars_Weather_Graphics.py*:

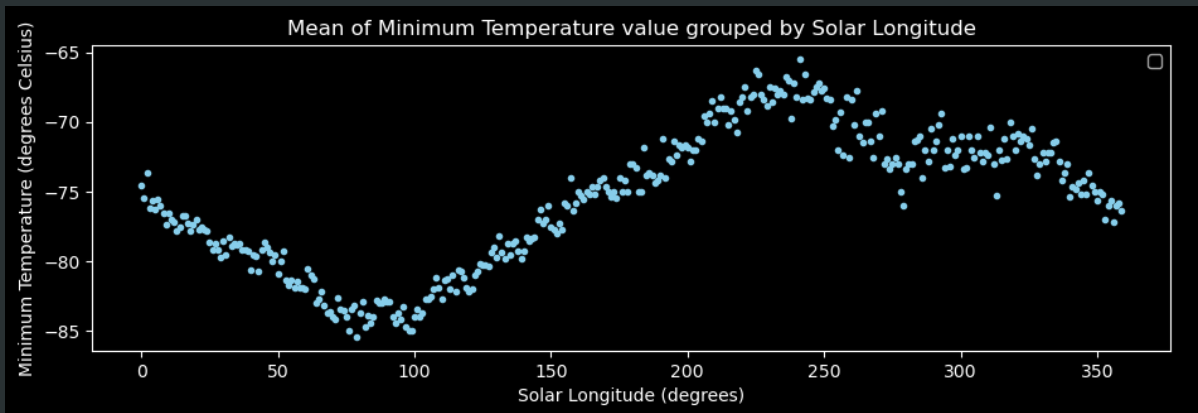


Figure 2: Minimum Temperature evolution as function of Solar Longitude

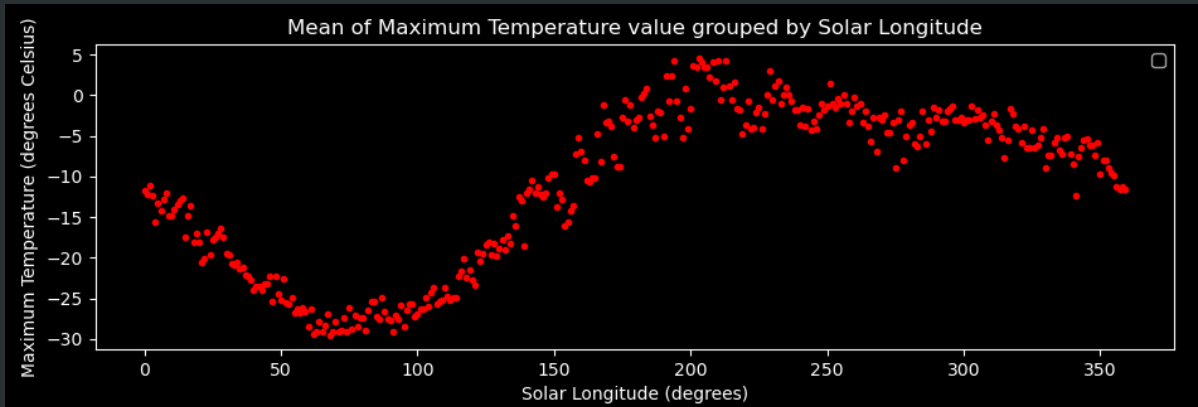


Figure 3: Maximum Temperature evolution as function of Solar Longitude

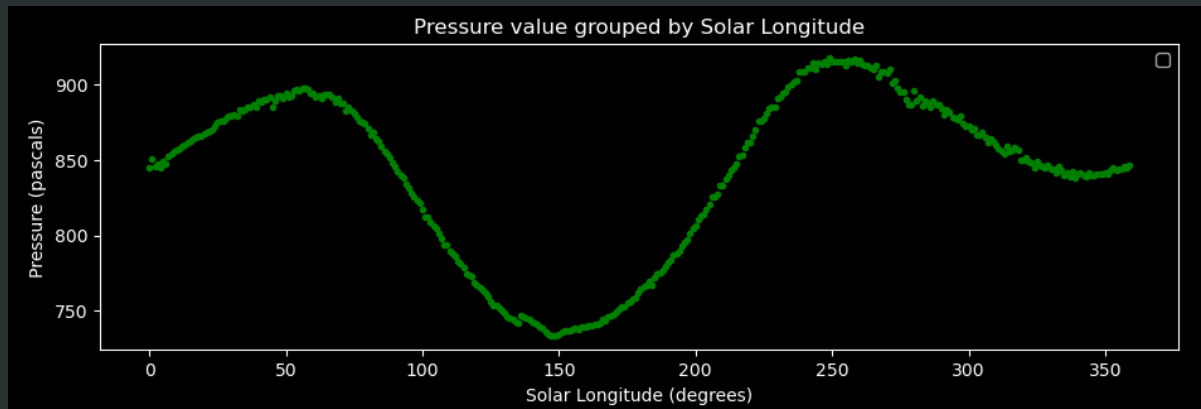


Figure 4: Pressure evolution as function of Solar Longitude

So already, this is pretty reassuring to see that these parameters appear to evolve consistently and logically throughout the whole Martian year. Maximum and Minimum temperature have the same behaviour, as expected because they reflect the same data. If we think in terms of season, the lower temperatures recorded could corresponds to Winter's solstice (arround $ls=90$ degrees), and the higher temperatures to Summer's solstice (arround 250 degrees).

On the other hand, it surprised me to see the pressure going up while temperatures are dropping down, at the beginning of the graphs (This doesn't seems in accordance with *ideal gas law*: $PV=nRT$). Apparently, this is usual on *Mars* because its trajectory arround the sun is more eccentric than *Earth* and most of the Martian ice contains *CO2* which has its own state cycle and affects the results. So the n parameter is actually also changing. But this is not the main topic of this CourseWork.

3.3 Parameters Evolution Sol per Sol

I also wondered how these meterological parameters changed throughout *Cu-riosity's* whole given samples by showing them sol per sol since it landed:

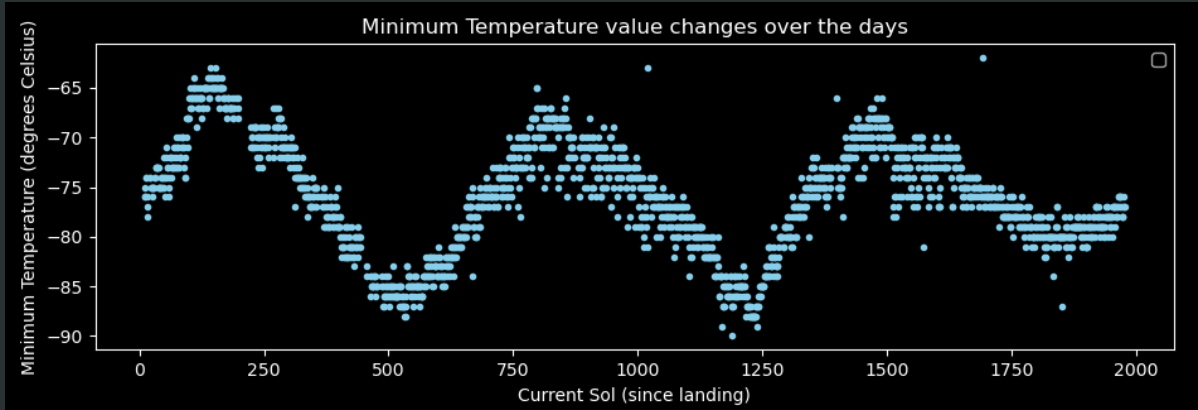


Figure 5: Minimum Temperature evolution graph by sol

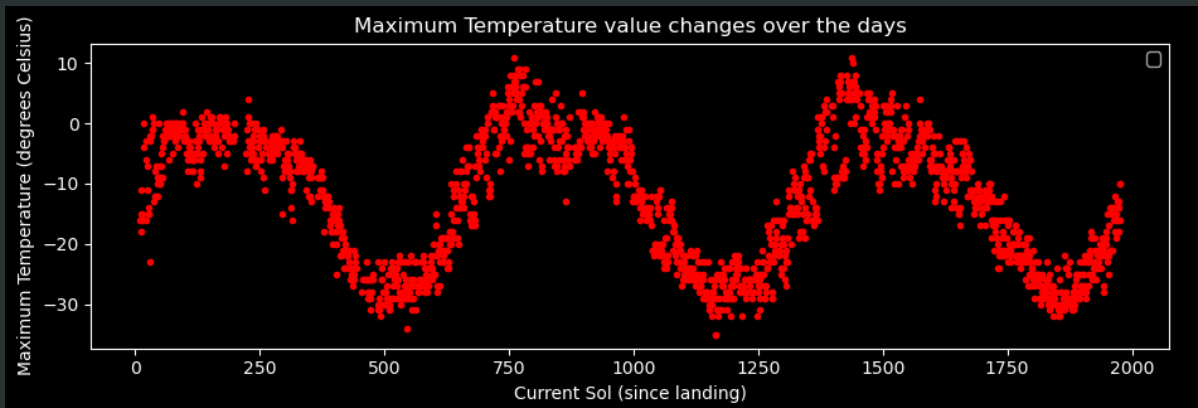


Figure 6: Maximum Temperature evolution graph by sol

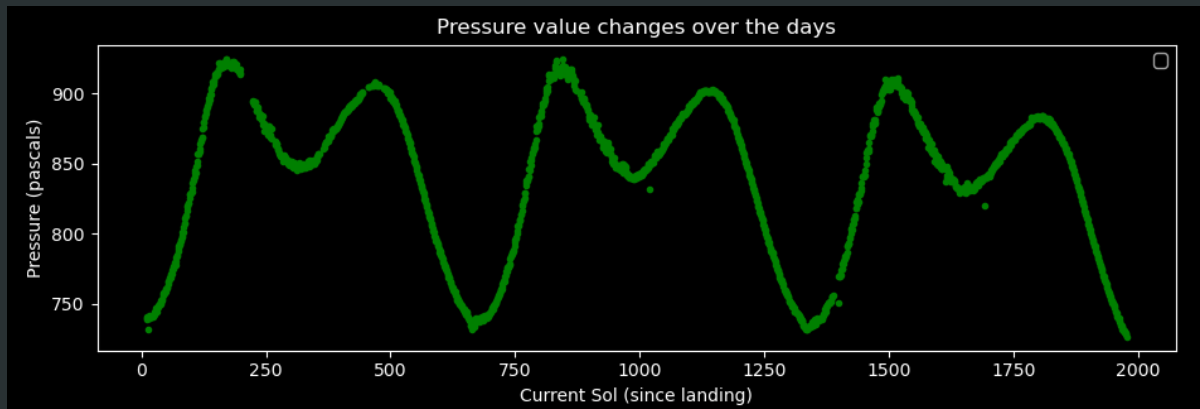


Figure 7: Pressure evolution graph by sol

These graphs cover even more time than the previous ones because the CSV file goes from day 1 to 1977 which is almost 3 Martian years. That's why we can see the same previous patterns but repeated 3 times on the new graphs for the Temperatures.

This is the same for Pressure, but I think this one is even more significant because of these two peaks repeating. But what I can't understand for now is why there are such huge pressure drops between each double-peak, which are not shown in the previous graph ?

Let's just assume that scales are different between these two graphs.

4 Clustering

4.1 Cluster Training

Since we assumed that these data may be classified into 4 categories: Summer, Autumn, Winter and Spring. Let's try to train a **Clustering** algorithm and see if its results are relevant if we indicates him 4 Cluster. So we are searching for a $4 \times 25\%$ repartition between these categories, showing 4 seasons. We can trust in it because the *mars-weather* CSV file seems to covers almost exactly 3 Martian years, so we have the same amount of data for each possible seasons.

Moving to *Weka*, I then opened *mars-weather* CSV file and kept these 5 categories: `sol`, `ls`, `min_temp`, `max_temp` and `pressure`.

By the way, we can visualize the same graphs than before:

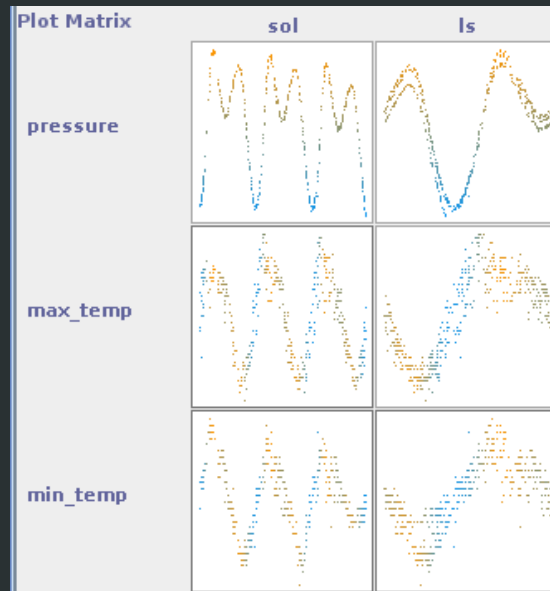


Figure 8: *Weka* Visualization Matrix

And I started my first **Clustering** using the *Kmeans* clusterer. First I've got a surprising high *Sum squarred errors* value, and one of these cluster contained 50% of all the data, which do not match with the 4 seasons concept. Then I realized the **sol** parameter is not characteristic of a sample and does not affect the weather condition (but teh results): Let's remove it, It was an obvious mistake to keep it.

So let's continue without `sol` parameter. Here is the results we got, using the same set for training and testing:

```

Number of iterations: 17
Within cluster sum of squared errors: 102.35829694907027
Missing values globally replaced with mean/mode

Cluster centroids:

```

Attribute	Full Data (1894)	Cluster# 0 (570)	1 (255)	2 (736)	3 (333)
ls	169.1806 +/-105.7385	47.4088 +/-26.7958	113.2314 +/-14.199	283.7446 +/-46.9213	167.2523 +/-22.987
min_temp	-76.121 +/-5.4647	-80.4105 +/-3.5511	-81.7832 +/-3.1104	-71.3214 +/-3.4204	-75.0511 +/-2.6321
max_temp	-12.5104 +/-10.6229	-22.4247 +/-6.1048	-23.5179 +/-4.0141	-3.6775 +/-4.7975	-6.6336 +/-6.8445
pressure	841.0664 +/-53.8649	875.2588 +/-18.5567	789.9231 +/-28.5716	870.0066 +/-29.5778	757.7387 +/-23.5491

```

Time taken to build model (full training data) : 0.02 seconds

=== Model and evaluation on training set ===

Clustered Instances
0      570 ( 30%)
1      255 ( 13%)
2      736 ( 39%)
3      333 ( 18%)

```

Figure 9: *Kmeans* Clustering within 4 categories

Moreover, normalizing, standardizing, or filling the missing data (there is 27 of them), surprisingly did not change the results: when parameters are chosen, repartition results remains all the same.

When we take a look at the **min_temp** line, it stays roughly the same across the four clusters. Thinking about it, we can assume minimal temperatures are reached the **night** on Mars, when there is no light. But temperature differences between seasons precisely depends on the sun rays repartition over the surface: **min_temp** is not a strong enough discriminating characteristic in order to know which season we are, but **max_temp** actually is.

Let's try another time without `min_temp`:

```

Number of iterations: 12
Within cluster sum of squared errors: 75.64412484373177
Missing values globally replaced with mean/mode

Cluster centroids:
Attribute      Full Data      Cluster#
                (1894)      0          1          2          3
                (582)      (247)      (729)      (336)
=====
ls             169.1806      48.2371      114.4656      284.5158      168.6577
                +/-105.7385  +/-27.3791  +/-14.5769  +/-46.4774  +/-21.4826

max_temp       -12.5104      -22.4812      -23.3768      -3.7622      -6.2321
                +/-10.6229  +/-6.0922  +/-3.9771  +/-4.7258  +/-6.8767

pressure        841.0664      874.4562      787.0057      870.5485      759.006
                +/-53.8649  +/-19.1956  +/-27.9158  +/-29.1929  +/-24.3867

Time taken to build model (full training data) : 0.01 seconds

=== Model and evaluation on training set ===

Clustered Instances
0          582 ( 31%)
1          247 ( 13%)
2          729 ( 38%)
3          336 ( 18%)

```

Figure 10: *Kmeans* Clustering within 4 categories (With no filters and by fulfilling the missing data with their means)

The repartition percentages are still not quite but are not aberrant. The *sum of squared error* is better though, so I think it is a great improvement considering 1894 instances and 4 cluster.

Let's not forget that temperatures are not the main characteristic of the seasons, sols are cold and sometimes cold, sometimes hot. These are extrem values that we can't remove and fully rely on. However, pressure seems to have a well defined behaviour which depends on solar longitude, but this is only my perception.

Besides, *Curiosity* sampled these values in the *Gale's Crater* which is almost equatorian, slightly to the south hemisphere. So seasons are more balanced than other locations and values in this dataset are unfortunately less significative.

So now, the question is: which cluster is which season ?

On earth, the origin of solar longitude, which is the *0 degrees* angle, is reached at the *Vernal Equinox* and each season got his own angle, as this table shows it:

Solar longitude	Date	Time (Earth days)
0°	11 April 1955	199
90°	27 October 1955	183
180°	27 April 1956	147
270°	21 September 1956	158
0°	26 February 1957	Total: 687

Figure 11: Recognisable solar longitude on *Earth* (*Wikipedia*)

This is pretty the same pattern and logic on *Mars*:

Seasons begin at 90 degree intervals of solar longitude (L_s) at equinoxes and solstices.^[10]

solar longitude (L_s)	event	months	northern hemisphere		southern hemisphere	
			event	season	event	seasons
0	northward equinox	1, 2, 3	vernal equinox	spring	autumnal equinox	autumn
90	northern solstice	4, 5, 6	summer solstice	summer	winter solstice	winter
180	southward equinox	7, 8, 9	autumnal equinox	autumn	vernal equinox	spring
270	southern solstice	10, 11, 12	winter solstice	winter	summer solstice	summer

Figure 12: Recognisable solar longitude on *Mars* (*Wikipedia*)

Using the results, It's still pretty difficult to put a season on each cluster ...

The hotter season (arround -3 degrees), which is probably summer, has a solar longitude which is not recognizable (it is supposed to be between 90 and 180 degrees) UNLESS we consider Curiosity to be in the southern hemisphere (which is actually true), so angles are inverted.

Considering this, let be each cluster become:

- **Spring:** Third category. *Actual value + standard deviation = 169 + 21 = 180 degrees*, so it could work.
- **Summer:** Second category. Between 270 and 360 degrees.
- **Autumn:** First category. Arround 0 and 90 degrees.
- **Winter:** Fourth category. Between 90 and 180 degrees.

Temperatures seems to match with each category too.

5 Predicting the Weather ?

After categorizing and recognizing the data, I started to wonder whether it could be used to predict future values. Of course, **Weather Forecasting** is a very complex discipline which requires expensive tools, advanced concepts, and powerful machines. That's why here, we will only limit ourselves to a sol-by-sol prediction.

The main goal is to imitate a **Long Short Term Memory (LSTM)** training model on *Weka*, as mentioned by miss *Priyadarshini* in her article.

5.1 LSTM Adapted CSV Maker Script

In order to make the **Long Short Term Memory** strategy work, we need to re-organize our data as follows: each line is a couple (*Data sol n-1*, *Data sol n*). At first, I tried to use the *TimesSeriesTranslate* filter proposed by *Weka* on the original CSV file, but it was rather tedious, require us to repeat the same actions, and I honestly did not understand how it works.

That's why I just typed a basic Python script adapting the original CSV file so it become readable by *Weka* **LSTM** strategies. You can find the source code in the *Python* repository as *LSTM_CSV_MAKER.py* and consult the generated file in the *CSV* as *mars-weather-LSTM_adapted.csv*.

But basically, It just looks like this:

```
1  sol_t-1,ls_t-1,max_temp_t-1,pressure_t-1,sol,ls,max_temp,pressure
2  10.0,155.0,-16.0,739.0,11,156,-11.0,740.0
3  11.0,156.0,-11.0,740.0,12,156,-18.0,741.0
4  12.0,156.0,-18.0,741.0,13,157,-15.0,732.0
5  13.0,157.0,-15.0,732.0,14,157,-16.0,740.0
6  14.0,157.0,-16.0,740.0,15,158,-15.0,740.0
7  15.0,158.0,-15.0,740.0,16,158,0.0,740.0
8  16.0,158.0,0.0,740.0,17,159,-4.0,742.0
9  17.0,159.0,-4.0,742.0,21,161,-3.0,741.0
10 21.0,161.0,-3.0,741.0,22,162,-6.0,742.0
11 22.0,162.0,-6.0,742.0,23,162,-16.0,741.0
12 23.0,162.0,-16.0,741.0,24,163,-7.0,742.0
13 24.0,163.0,-7.0,742.0,25,163,-11.0,743.0
14 25.0,163.0,-11.0,743.0,26,164,-14.0,745.0
15 26.0,164.0,-14.0,745.0,27,164,-15.0,743.0
16 27.0,164.0,-15.0,743.0,28,165,-15.0,745.0
```

Figure 13: A sample of the *mars-weather-LSTM_adapted.csv* file

The **sol_t-1** and **sol_t** will not be used in the training since they only are benchmarks for guidance. Same for **ls_t-1**. However, **ls** is the only *n* level parameter which is relevant for predicting *n* level data. So we are keeping it.

5.2 *Weka* adapted LSTM strategy

Now, we just need to open the file, use the **Classify** section and choose *M5P*, in *trees* section. Each time we run our model, It will provide these kind of output:

```
LM num: 2
pressure =
    0.0777 * max_temp_t-1
    + 0.993 * pressure_t-1
    - 0.0058 * ls
    + 8.065

Number of Rules : 2

Time taken to build model: 0.06 seconds

=== Evaluation on training set ===
=== Summary ===

Correlation coefficient          0.9993
Mean absolute error             1.2808
Root mean squared error         2.0272
Relative absolute error         2.8548 %
Root relative squared error     3.7401 %
Total Number of Instances      1866
```

Figure 14: Output Example

”*LM*” is a node followed by its number, and its prediction, which can be either a constant or an equation. The main statistics are showed then.

Depending on chosen parameters, the produced trees and nodes are slightly different, and results too. But not from a great distance.

Weka suggests us 4 parameters:

- **Regression Tree Builder:** if marked, will not associates a Linear Regression Equation to each node, but a constant.
- **Minimum Number of Instances:** the more this value is high, the earlier the training tree will stop. So It is better to keep it low, but not too low in order to avoid **Overfitting** (set to a default value 4 here).
- **Prunning:** if marked, will erase branch considered as not relevant. It is entirely on *Weka*’s own though.
- **Smoothing:** if marked, will mix some parent-child nodes in order to control the ammount of nodes and to simplify the found equations.

I think this is better not to use **Regression Tree Builder** since we need the previous data to be used for predicting and besides, we've got a very limited ammount of nodes (only 2 rules) when I started using it:

```

pressure_t-1 <= 816.5 : LM1 (531/2.586%)
pressure_t-1 > 816.5 : LM2 (1335/4.11%)

LM num: 1
pressure =
    0.2777 * ls_t-1
    + 0.0214 * max_temp_t-1
    + 1.0026 * pressure_t-1
    - 0.2508 * ls
    - 5.5624

LM num: 2
pressure =
    0.0777 * max_temp_t-1
    + 0.993 * pressure_t-1
    - 0.0058 * ls
    + 8.065

Number of Rules : 2

```

Figure 15: Underfitted Tree for Pressure Prediction

I did not use the **Prunning** feature too. Because the data does vary very quickly, I think *Weka* is ignoring too much of them. Now we've got 56 rules. Using **Smoothing** feature just slightly decreased margin error, so we are gonna use It.

Number of Rules : 56		Number of Rules : 56	
Time taken to build model: 0.04 seconds		Time taken to build model: 0.05 seconds	
=== Evaluation on training set ===		=== Evaluation on training set ===	
=== Summary ===		=== Summary ===	
Correlation coefficient	0.9992	Correlation coefficient	0.9991
Mean absolute error	1.5293	Mean absolute error	1.7941
Root mean squared error	2.1598	Root mean squared error	2.3242
Relative absolute error	3.4087 %	Relative absolute error	3.999 %
Root relative squared error	3.9848 %	Root relative squared error	4.2881 %
Total Number of Instances	1866	Total Number of Instances	1866

Figure 16: With and without **Smoothing** feature comparison for Pressure (Left to right)

5.3 Final Results and Testing Phase

So, the final and best results I've got for both pressure and temperature prediction are:

LM num: 56 pressure = 0.0044 * max_temp_t-1 + 0.4092 * pressure_t-1 - 0.0002 * ls + 543.5251	LM num: 773 max_temp = - 0.5
Number of Rules : 56	LM num: 774 max_temp = + 0
Time taken to build model: 0.05 seconds	Number of Rules : 774
Time taken to build model: 0.09 seconds	
=== Evaluation on training set ===	=== Evaluation on training set ===
=== Summary ===	=== Summary ===
Correlation coefficient 0.9992	Correlation coefficient 0.9864
Mean absolute error 1.5293	Mean absolute error 1.1501
Root mean squared error 2.1598	Root mean squared error 1.7592
Relative absolute error 3.4087 %	Relative absolute error 12.1708 %
Root relative squared error 3.9848 %	Root relative squared error 16.4424 %
Total Number of Instances 1866	Total Number of Instances 1866

Figure 17: LSTM Final Results

Surprisingly, results are better for temperature when they are predicted as constants and only by using tree rules, which uses different parameters:

```

ls <= 139.5 :
| max_temp_t-1 <= -22.5 :
| | max_temp_t-1 <= -25.5 :
| | | max_temp_t-1 <= -28.5 :
| | | | pressure <= 891.5 :
| | | | | pressure <= 876.5 :
| | | | | | ls <= 91.5 :
| | | | | | | ls <= 81.5 :
| | | | | | | | pressure_t-1 <= 872.5 :
| | | | | | | | | max_temp_t-1 <= -30.5 :
| | | | | | | | | | max_temp_t-1 <= -31.5 : LM1 (3/7.631%)
| | | | | | | | | | max_temp_t-1 > -31.5 : LM2 (2/14.02%)
| | | | | | | | | | max_temp_t-1 > -30.5 :
| | | | | | | | | | | pressure_t-1 <= 857.5 : LM3 (3/4.406%)
| | | | | | | | | | | pressure_t-1 > 857.5 :
| | | | | | | | | | | | pressure_t-1 <= 870 :
| | | | | | | | | | | | | pressure_t-1 <= 865 : LM4 (3/11.
| | | | | | | | | | | | | pressure_t-1 > 865 :
| | | | | | | | | | | | | | max_temp_t-1 <= -29.5 : LM5
| | | | | | | | | | | | | | max_temp_t-1 > -29.5 : LM6
| | | | | | | | | | | | | | | pressure_t-1 > 870 : LM7 (2/14.02%)
| | | | | | | | | | | | | | | pressure_t-1 > 872.5 :
| | | | | | | | | | | | | | | | max_temp_t-1 <= -29.5 : LM8 (3/20.191%)
| | | | | | | | | | | | | | | | max_temp_t-1 > -29.5 : LM9 (2/0%)
| | | | | | | | | | | | | | | | ls > 81.5 :
| | | | | | | | | | | | | | | | | pressure_t-1 <= 839 :

```

Figure 18: Sample of the Tree Rules for Temperature on *Weka*

I am pretty surprised to see this tree using **pressure** for predicting **max.temp** ... But unfortunately I can not change that.

Seeing 774 rules made me fear that our model was overfitted, but it doesn't seem to be according to results.

Here is also a sample of what the testing phase looks like on Weka:

```
Time taken to build model: 0.08 seconds

=== Predictions on training set ===

inst#,    actual, predicted, error
1      -11      -11.5      -0.5
2      -18      -17.5       0.5
3      -15      -15.5      -0.5
4      -16       -8         8
5      -15      -12         3
6       0       -8        -8
7       -4      -0.667     3.333
8       -3      -2.5       0.5
9       -6       -6         0
10      -16     -11.667     4.333
11       -7     -7.667    -0.667
12      -11     -11.667    -0.667
13      -14     -14         0
14      -15     -14         1
15      -15     -14.667     0.333
16       -2       -2         0
17       -3     -2.667     0.333
18      -23     -11.667    11.333
19       -2       -2         0
20       1       0.5      -0.5
21      -1       -1         0
22      -1      -1.6      -0.6
23       0        1         1
24      -13     -11.5       1.5
25       -8     -10.333    -2.333
26      -12     -12         0
```

Figure 19: Sample of Tested Temperatures Instances by Weka

As we can see, most of them are pretty good, and other are mistaken by more than 10 degrees.

Instances between 2 and 8 were actually empty on CSV file and has been completed by *Weka*, maybe should I have simply removed them.

6 Conclusion

The given results seems quite and may appear modest, but they above all have been produced by approaches and procedures I've carfully tried to well thought-out and that I understand.

This CourseWork allowed me to better understand severals new tools and concepts that were previously unfamiliar to me, by actively manipulating them and by searching their details.

I tried to extract the most informations I can gain from that CSV file by compiling them and training predictive models, but of course, it goes without saying that you will always be able to be more precise, and to produce more and better findings.

7 References

- **Research Gate**, "*Mars weather data analysis using machine learning techniques*" by *Ishaani Priyadarshini*, last edited on December 2021: https://www.researchgate.net/publication/352976885_Mars_weather_data_analysis_using_machine_learning_techniques - **Wikipedia**, "*Atmosphere of Mars*", section "*Current chemical composition*", last edited on April 2025: https://en.wikipedia.org/wiki/Atmosphere_of_Mars?#Current_chemical_composition
- **Wikipedia**, "*Solar longitude (Earth)*", last edited on November 2023: https://en.wikipedia.org/wiki/Solar_longitude
- **Wikipedia**, "*Timekeeping on Mars*", section "*Definition of Year and Seasons*", last edited on April 2025: https://en.wikipedia.org/wiki/Timekeeping_on_Mars#Definition_of_year_and_seasons
- **Kanaries**, "*How to Use the Pandas Shift Method for Data Analysis*" by *Omar C. Williams*, last edited on August 2023: <https://docs.kanaries.net/topics/Pandas/pandas-shift>